MATH10017: R Programming Exercises

Contents

Short Questions **1**
- Question 1: Vectors and indexing . . . 1
- Question 2: Vectorised computation . . . 2
- Question 3: Matrices and row/column operations . . . 2
- Question 4: Writing a simple function . . . 2
- Question 5: Using `apply` instead of a loop . . . 3
- Question 6: Data frames: creating, filtering, selecting . . . 3
- Question 7: Graphics in base R . . . 4
- Question 8: Lists and `sapply` . . . 5
- Question 9: Logical indexing with `my_which` . . . 6

Longer Questions **6**
- Question 1: Matrix operations and linear algebra . . . 6
- Question 2: Writing and using your own function with `sapply` . . . 8
- Question 3: Data frames and summary statistics . . . 8
- Question 4: Base graphics with multiple layers . . . 9
- Question 5: Using `apply` on a matrix of statistics . . . 10
- Question 6 (advanced): Functional programming with `lapply` and data frames . . . 11
- Question 7: Vector creation, indexing, and summary . . . 13
- Question 8: Working with sequences and proportions . . . 14
- Question 9: Vectorised unit conversions and labels . . . 14
- Question 10: Recycling . . . 15
- Question 11: Matrix creation and column summaries . . . 15
- Question 12: Submatrices and summary statistics . . . 16
- Question 13: Z-score function . . . 17
- Question 14: Weighted mean function . . . 17

Short Questions

Question 1: Vectors and indexing

Create a numeric vector `x` containing the integers from 1 to 20.
Using *one line of code*, extract all elements of `x` that are multiples of 3 and store them in `y`.

Solution

```
x <- 1:20
y <- x[x %% 3 == 0]
y
```

```
## [1]  3  6  9 12 15 18
```

Question 2: Vectorised computation

You have a vector of temperatures in Celsius:

```
temp_C <- c(0, 10, 20, 30)
```

Write *vectorised* R code (no explicit `for` loop) to convert these to Fahrenheit and store them in `temp_F`.
(Use the formula F = 9/5 * C + 32.)

Solution

```
temp_C <- c(0, 10, 20, 30)
temp_F <- 9/5 * temp_C + 32
temp_F
```

```
## [1] 32 50 68 86
```

Question 3: Matrices and row/column operations

Create a 3×4 matrix `M` containing the integers from 1 to 12 (filled by rows).

Then:

1. Compute the sum of each row (a length-3 vector).
2. Extract the second column of `M` as a vector.

Solution

```
M <- matrix(1:12, nrow = 3, ncol = 4, byrow = TRUE)
M
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    2    3    4
## [2,]    5    6    7    8
## [3,]    9   10   11   12
```

```
row_sums <- rowSums(M)
second_col <- M[, 2]

row_sums
```

```
## [1] 10 26 42
```

```
second_col
```

```
## [1]  2  6 10
```

Question 4: Writing a simple function

Write an R function `my_mean` that takes a numeric vector `x` and returns its mean **without** using the built-in `mean()` function.
(You may assume `x` has length > 0.)

Solution

```
my_mean <- function(x) {
  sum(x) / length(x)
}

my_mean(c(1, 2, 3))
```

```
## [1] 2
```

Question 5: Using `apply` instead of a loop

Let

```
A <- matrix(1:9, nrow = 3)  # 3×3 matrix
```

Use `apply()` to compute the **sum of each column** of `A` (i.e. a length-3 vector) and store it in `col_sums`. Do not use `colSums()`.

Solution

```
A <- matrix(1:9, nrow = 3)
A
```

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

```
col_sums <- apply(A, 2, sum)
col_sums
```

```
## [1]  6 15 24
```

Question 6: Data frames: creating, filtering, selecting

Create a data frame `students` with three columns:

- `name` = `c("Alice", "Bob", "Cara", "Dan")`
- `age` = `c(19, 20, 18, 21)`
- `mark` = `c(72, 55, 88, 64)`

Then:

1. Select only the rows for students with `mark >= 60`.
2. From that filtered data frame, extract only the `name` and `mark` columns.

Solution

```
students <- data.frame(
  name = c("Alice", "Bob", "Cara", "Dan"),
```

```
  age  = c(19, 20, 18, 21),
  mark = c(72, 55, 88, 64)
)
students
```

```
##    name age mark
## 1 Alice  19   72
## 2   Bob  20   55
## 3  Cara  18   88
## 4   Dan  21   64
```

```
pass_df <- students[students$mark >= 60, ]
result  <- pass_df[, c("name", "mark")]

pass_df
```

```
##    name age mark
## 1 Alice  19   72
## 3  Cara  18   88
## 4   Dan  21   64
```

```
result
```

```
##    name mark
## 1 Alice   72
## 3  Cara   88
## 4   Dan   64
```

Question 7: Graphics in base R

Given:

```
x <- 0:10
y <- x^2
```

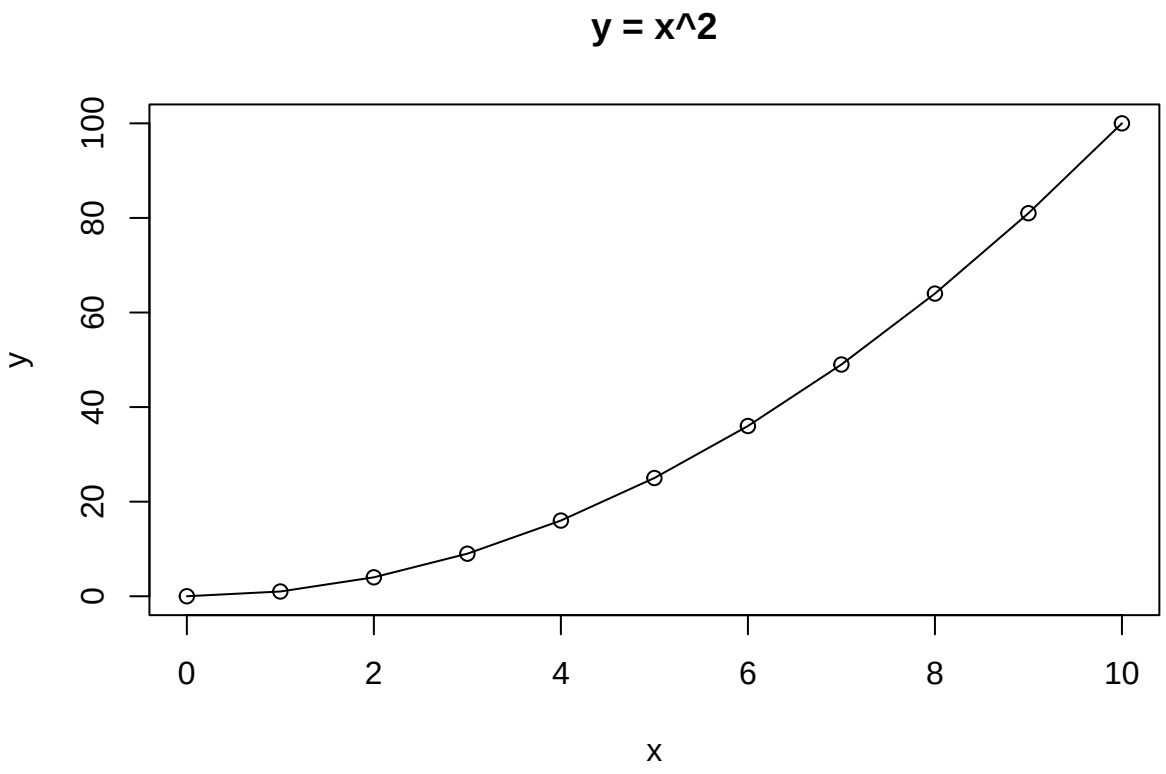
Write R code to:

1. Plot `y` against `x` using points.
2. Add a **line** connecting the points on the same plot.

Solution

```
x <- 0:10
y <- x^2

plot(x, y, main = "y = x^2", xlab = "x", ylab = "y")
lines(x, y)
```



Question 8: Lists and `sapply`

Consider the list:

```
L <- list(
  a = 1:3,
  b = 10:15,
  c = c(5, 5, 5, 5)
)
```

Use `sapply()` to compute the **length** of each element of `L`, returning a numeric vector of lengths.

Solution

```
L <- list(
  a = 1:3,
  b = 10:15,
  c = c(5, 5, 5, 5)
)

len_vec <- sapply(L, length)
len_vec
```

```
## a b c
## 3 6 4
```

Question 9: Logical indexing with `my_which`

Create a function `my_which` that takes as input a logical vector `v` and returns the indices of all `TRUE` elements (similar to R's built-in `which()` function).

Then test your function on:

```
v <- c(TRUE, FALSE, TRUE, TRUE, FALSE)
```

Then consider the numeric vector:

```
z <- c(3, 8, 1, 8, 5)
```

Use `my_which()` to find the **indices** of all elements equal to 8, and store them in `idx`.
Then use `idx` to replace those elements with 0 (modifying `z`).

Solution

```
my_which <- function(v) {
  out <- 1:length(v)
  return( out[v] )
}

v <- c(TRUE, FALSE, TRUE, TRUE, FALSE)
my_which(v)
```

```
## [1] 1 3 4
```

```
which(v)
```

```
## [1] 1 3 4
```

```
z <- c(3, 8, 1, 8, 5)
z
```

```
## [1] 3 8 1 8 5
```

```
idx <- my_which(z == 8)
idx
```

```
## [1] 2 4
```

```
z[idx] <- 0
z
```

```
## [1] 3 0 1 0 5
```

Longer Questions

Question 1: Matrix operations and linear algebra

Let

```
A <- matrix(c(1, 2, 3,
              0, 1, 4,
              5, 6, 0), nrow = 3, byrow = TRUE)
b <- c(7, 4, 10)
```

1. Compute the transpose of `A`.

2. Compute the matrix product `A %*% b`.

3. Solve the linear system `A x = b` for `x` by computing $x = A^{-1}b$ (use `solve()`).

Solution

```
A <- matrix(c(1, 2, 3,
              0, 1, 4,
              5, 6, 0), nrow = 3, byrow = TRUE)
b <- c(7, 4, 10)

A
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    0    1    4
## [3,]    5    6    0
```

```
b
```

```
## [1]  7  4 10
```

```
# 1. Transpose
A_t <- t(A)
A_t
```

```
##      [,1] [,2] [,3]
## [1,]    1    0    5
## [2,]    2    1    6
## [3,]    3    4    0
```

```
# 2. Matrix product
Ab <- A %*% b
Ab
```

```
##      [,1]
## [1,]   45
## [2,]   44
## [3,]   59
```

```
# 3. Solve A x = b
x <- solve(A) %*% b
x
```

```
##      [,1]
## [1,]  -46
## [2,]   40
## [3,]   -9
```

```
# Check: A %*% x should be b
A %*% x
```

```
##      [,1]
## [1,]    7
## [2,]    4
## [3,]   10
```

Question 2: Writing and using your own function with `sapply`

Write a function `is_prime` that takes a single positive integer `n` and returns `TRUE` if `n` is prime, `FALSE` otherwise.

Then:

1. Use `is_prime` on the numbers from 1 to 20.
2. Use `sapply(1:20, is_prime)` to get a logical vector of primality.
3. Use this logical vector to extract all primes between 1 and 20.

(You may use a simple, not highly optimised, algorithm.)

Solution

```
is_prime <- function(n) {
  if (n <= 1) return(FALSE)
  if (n <= 3) return(TRUE)
  for (k in 2:floor(sqrt(n))) {
    if (n %% k == 0) return(FALSE)
  }
  TRUE
}

# 1 & 2. Apply to 1:20
n_vals <- 1:20
prime_flags <- sapply(n_vals, is_prime)
prime_flags
```

```
##  [1] FALSE  TRUE  TRUE FALSE  TRUE FALSE  TRUE FALSE FALSE FALSE  TRUE FALSE
## [13]  TRUE FALSE FALSE FALSE  TRUE FALSE  TRUE FALSE
```

```
# 3. Extract the primes
primes <- n_vals[prime_flags]
primes
```

```
## [1]  2  3  5  7 11 13 17 19
```

Question 3: Data frames and summary statistics

You have exam data for a class:

```
students <- data.frame(
  name   = c("Alice", "Bob", "Cara", "Dan", "Eli"),
  gender = c("F", "M", "F", "M", "M"),
  mark1  = c(72, 55, 88, 64, 90),
  mark2  = c(68, 60, 91, 70, 85)
)
```

1. Add a new column `avg_mark` which is the mean of `mark1` and `mark2` for each student.
2. Find the **overall** mean of `avg_mark`.
3. Use logical indexing to extract the rows of students with `avg_mark >= 70`.

Solution

```
students <- data.frame(
  name   = c("Alice", "Bob", "Cara", "Dan", "Eli"),
  gender = c("F", "M", "F", "M", "M"),
  mark1  = c(72, 55, 88, 64, 90),
  mark2  = c(68, 60, 91, 70, 85)
)
students
```

```
##    name gender mark1 mark2
## 1 Alice      F    72    68
## 2   Bob      M    55    60
## 3  Cara      F    88    91
## 4   Dan      M    64    70
## 5   Eli      M    90    85
```

```
# 1. Add avg_mark
students$avg_mark <- (students$mark1 + students$mark2) / 2
students
```

```
##    name gender mark1 mark2 avg_mark
## 1 Alice      F    72    68     70.0
## 2   Bob      M    55    60     57.5
## 3  Cara      F    88    91     89.5
## 4   Dan      M    64    70     67.0
## 5   Eli      M    90    85     87.5
```

```
# 2. Overall mean
overall_mean <- mean(students$avg_mark)
overall_mean
```

```
## [1] 74.3
```

```
# 3. Students with avg_mark >= 70
good_students <- students[students$avg_mark >= 70, ]
good_students
```

```
##    name gender mark1 mark2 avg_mark
## 1 Alice      F    72    68     70.0
## 3  Cara      F    88    91     89.5
## 5   Eli      M    90    85     87.5
```

Question 4: Base graphics with multiple layers

Suppose you have data from a simple physics experiment:

```
time <- seq(0, 10, by = 0.5)
distance <- 3 * time + rnorm(length(time), mean = 0, sd = 2)
```

1. Plot `distance` (y-axis) against `time` (x-axis) using points.

2. Add a straight line showing the **true** underlying relationship `distance = 3 * time` in a different plotting layer.

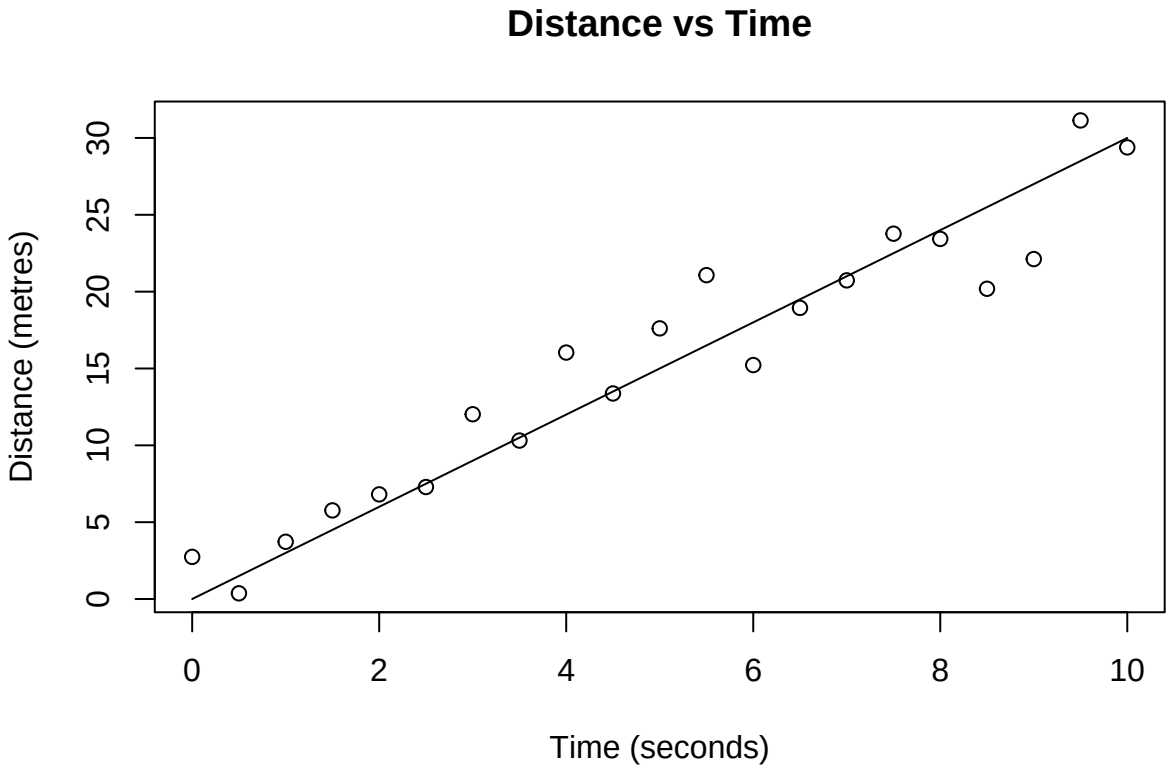
3. Add a main title and axis labels.

Solution

```r
set.seed(42)
time <- seq(0, 10, by = 0.5)
distance <- 3 * time + rnorm(length(time), mean = 0, sd = 2)

# 1. Scatter plot
plot(time, distance,
     main = "Distance vs Time",
     xlab = "Time (seconds)",
     ylab = "Distance (metres)")

# 2. True underlying line
lines(time, 3 * time)
```



Question 5: Using `apply` on a matrix of statistics

Create a 4×5 matrix `M` containing the integers 1 to 20 (filled by rows).

Then:

1. Use `apply()` to find the **mean of each row**.
2. Use `apply()` to find the **maximum of each column**.
3. Combine these results into a list with named components `row_means` and `col_max`.

Solution

```
M <- matrix(1:20, nrow = 4, ncol = 5, byrow = TRUE)
M
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    2    3    4    5
## [2,]    6    7    8    9   10
## [3,]   11   12   13   14   15
## [4,]   16   17   18   19   20
```

```
# 1. Row means
row_means <- apply(M, 1, mean)
row_means
```

```
## [1]  3  8 13 18
```

```
# 2. Column maxima
col_max <- apply(M, 2, max)
col_max
```

```
## [1] 16 17 18 19 20
```

```
# 3. Combine into a list
summary_list <- list(
  row_means = row_means,
  col_max   = col_max
)
summary_list
```

```
## $row_means
## [1]  3  8 13 18
##
## $col_max
## [1] 16 17 18 19 20
```

Question 6 (advanced): Functional programming with `lapply` and data frames

This is an **advanced** question.

Create a list `L` containing three small data frames, each with a single numeric column:

```
set.seed(1)
L <- list(
  df1 = data.frame(x = rnorm(5, mean = 0, sd = 1)),
  df2 = data.frame(x = rnorm(5, mean = 10, sd = 2)),
  df3 = data.frame(x = rnorm(5, mean = -5, sd = 1))
)
```

1. Use `lapply()` to compute the mean of column `x` for each data frame in the list.

2. Convert the result into a numeric vector using `unlist()`.

3. Add a new column `x_centered` to each data frame, where you subtract its own mean from each value of `x`. (Hint: use `lapply()` again.)

Solution

```
set.seed(1)
L <- list(
  df1 = data.frame(x = rnorm(5, mean = 0, sd = 1)),
  df2 = data.frame(x = rnorm(5, mean = 10, sd = 2)),
  df3 = data.frame(x = rnorm(5, mean = -5, sd = 1))
)
L
```

```
## $df1
##           x
## 1 -0.6264538
## 2  0.1836433
## 3 -0.8356286
## 4  1.5952808
## 5  0.3295078
##
## $df2
##          x
## 1  8.359063
## 2 10.974858
## 3 11.476649
## 4 11.151563
## 5  9.389223
##
## $df3
##          x
## 1 -3.488219
## 2 -4.610157
## 3 -5.621241
## 4 -7.214700
## 5 -3.875069
```

```
# 1. Means of x for each data frame
means_list <- lapply(L, function(d) mean(d$x))
means_list
```

```
## $df1
## [1] 0.1292699
##
## $df2
## [1] 10.27027
##
## $df3
## [1] -4.961877
```

```
# 2. Convert to numeric vector
means_vec <- unlist(means_list)
means_vec
```

```
##        df1        df2        df3
##  0.1292699 10.2702713 -4.9618770
```

```
# 3. Add x_centered using lapply
L_centered <- lapply(L, function(d) {
```

```
  m <- mean(d$x)
  d$x_centered <- d$x - m
  d
})

L_centered
```

```
## $df1
##            x  x_centered
## 1 -0.6264538 -0.75572371
## 2  0.1836433  0.05437343
## 3 -0.8356286 -0.96489851
## 4  1.5952808  1.46601091
## 5  0.3295078  0.20023788
##
## $df2
##           x x_centered
## 1  8.359063 -1.9112081
## 2 10.974858  0.7045868
## 3 11.476649  1.2063781
## 4 11.151563  0.8812914
## 5  9.389223 -0.8810481
##
## $df3
##           x x_centered
## 1 -3.488219  1.4736582
## 2 -4.610157  0.3517203
## 3 -5.621241 -0.6593636
## 4 -7.214700 -2.2528229
## 5 -3.875069  1.0868079
```

Question 7: Vector creation, indexing, and summary

You are given daily production outputs (in kg) from a small factory over 30 days, stored as a sequence from 12 to 41:

```
prod <- 12:41
```

1. Extract all production values greater than 30.
2. Count how many days exceeded 30 kg production.
3. Compute the mean production **only among the days where production was greater than 30**.

Solution

```
prod <- 12:41

high <- prod[prod > 30]
high
```

```
##  [1] 31 32 33 34 35 36 37 38 39 40 41
```

```r
count_high <- length(high)
count_high
```

```
## [1] 11
```

```r
mean_high <- mean(high)
mean_high
```

```
## [1] 36
```

Question 8: Working with sequences and proportions

Create a vector `x` containing 100 values equally spaced from -π to +π (`pi` in R).

1. Print the first 6 and last 6 elements.
2. Use logical indexing to extract values of `x` between -1 and +1.
3. Compute the proportion of values in this interval.

Solution

```r
x <- seq(from = -pi, to = pi, length.out = 100)

x[1:6]
```

```
## [1] -3.141593 -3.078126 -3.014660 -2.951193 -2.887727 -2.824260
```

```r
x[95:100]
```

```
## [1] 2.824260 2.887727 2.951193 3.014660 3.078126 3.141593
```

```r
mid <- x[x >= -1 & x <= 1]
mid
```

```
##  [1] -0.98373103 -0.92026451 -0.85679800 -0.79333148 -0.72986496 -0.66639844
##  [7] -0.60293192 -0.53946541 -0.47599889 -0.41253237 -0.34906585 -0.28559933
## [13] -0.22213281 -0.15866630 -0.09519978 -0.03173326  0.03173326  0.09519978
## [19]  0.15866630  0.22213281  0.28559933  0.34906585  0.41253237  0.47599889
## [25]  0.53946541  0.60293192  0.66639844  0.72986496  0.79333148  0.85679800
## [31]  0.92026451  0.98373103
```

```r
prop_mid <- length(mid) / length(x)
prop_mid
```

```
## [1] 0.32
```

Question 9: Vectorised unit conversions and labels

The vector below represents raw heights measured in **metres** for a sample of 6 people:

```r
h <- c(1.55, 1.72, 1.68, 1.81, 1.60, 1.90)
```

1. Convert these heights to **centimetres**.

2. Convert them to **inches** (1 inch = 2.54 cm).

3. Create a vector indicating `"tall"` for heights > 1.75 m and `"not tall"` otherwise.

Solution

```
h <- c(1.55, 1.72, 1.68, 1.81, 1.60, 1.90)

cm <- h * 100
cm
```

```
## [1] 155 172 168 181 160 190
```

```
inches <- cm / 2.54
inches
```

```
## [1] 61.02362 67.71654 66.14173 71.25984 62.99213 74.80315
```

```
label <- character(length(h))
label[h > 1.75] <- "tall"
label[!(h > 1.75)] <- "not tall"
label
```

```
## [1] "not tall" "not tall" "not tall" "tall"     "not tall" "tall"
```

Question 10: Recycling

Consider:

```
x <- c(3, 6, 9, 12, 15, 18)
y <- c(1, 2)
```

1. Compute `x + y`.

2. Explain in a few sentences how R handled the shorter vector `y` during this operation.

Solution

```
x <- c(3, 6, 9, 12, 15, 18)
y <- c(1, 2)

result <- x + y
result
```

```
## [1]  4  8 10 14 16 20
```

Explanation (to write in words): R automatically recycles `y` to the length of `x`, turning it into `c(1, 2, 1, 2, 1, 2)` and then adds element-wise.

Question 11: Matrix creation and column summaries

Create a **5×4 matrix** `M` that contains the integers 1 to 20 (filled by **rows**).

1. Extract the **first and third rows** of `M`.

2. Compute the **sum of each column** using `apply()` (do not use `colSums()`).

3. Multiply the matrix by 2 (vectorised multiplication).

Solution

```
M <- matrix(1:20, nrow = 5, ncol = 4, byrow = TRUE)
M
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    2    3    4
## [2,]    5    6    7    8
## [3,]    9   10   11   12
## [4,]   13   14   15   16
## [5,]   17   18   19   20
```

```
M[c(1, 3), ]
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    2    3    4
## [2,]    9   10   11   12
```

```
col_sums <- apply(M, 2, sum)
col_sums
```

```
## [1] 45 50 55 60
```

```
M2 <- M * 2
M2
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    2    4    6    8
## [2,]   10   12   14   16
## [3,]   18   20   22   24
## [4,]   26   28   30   32
## [5,]   34   36   38   40
```

Question 12: Submatrices and summary statistics

Using the matrix `M` from:

```
M <- matrix(rnorm(5*4), nrow = 5, ncol = 4, byrow = TRUE)
M
```

```
##             [,1]        [,2]        [,3]        [,4]
## [1,] -0.04493361 -0.01619026  0.94383621  0.82122120
## [2,]  0.59390132  0.91897737  0.78213630  0.07456498
## [3,] -1.98935170  0.61982575 -0.05612874 -0.15579551
## [4,] -1.47075238 -0.47815006  0.41794156  1.35867955
## [5,] -0.10278773  0.38767161 -0.05380504 -1.37705956
```

1. Extract the **submatrix of rows 3-4 and columns 2-3**.

2. Square each element of this submatrix and call the resulting matrix V.

3. Compute the inverse of $C = VV$.

Solution

```
sub <- M[3:4, 2:3]
sub
```

```
##            [,1]        [,2]
## [1,]  0.6198257 -0.05612874
## [2,] -0.4781501  0.41794156
```

```
V <- sub^2

solve(V%*%V)
```

```
##            [,1]       [,2]
## [1,]   7.086435 -0.3994895
## [2,] -28.990999 33.6531008
```

Question 13: Z-score function

Write a function `z_score()` that calculates the vector

$$z = \frac{x - \mu}{\sigma}$$

where `x` is a numeric vector, and `mu` and `sigma` are scalars.

1. Apply the function to `x <- c(10, 12, 9, 11, 15)` using `mu = 11` and `sigma = 2`.

2. Print the result on the R console.

Solution

```
z_score <- function(x, mu, sigma) {
  (x - mu) / sigma
}

x <- c(10, 12, 9, 11, 15)
z <- z_score(x, mu = 11, sigma = 2)
z
```

```
## [1] -0.5  0.5 -1.0  0.0  2.0
```

Question 14: Weighted mean function

Create a function `weighted_mean(x, w)` that computes the weighted mean of a numeric vector `x` with weights `w` using the formula:

$$\text{weighted mean} = \frac{\sum_{i=1}^{d} x_i w_i}{\sum_{j=1}^{d} w_j}$$

where d is the length of `x` and `w`.

1. Test it on

```r
x <- c(60, 75, 80)
w <- c(1, 2, 1)
```

2. Check the result of `weighted_mean(x, w)` against those of `weighted.mean(x, w)` (R default function for doing this).

Solution

```r
weighted_mean <- function(x, w) {
  sum(x * w) / sum(w)
}

x <- c(60, 75, 80)
w <- c(1, 2, 1)

weighted_mean(x, w)
```

```
## [1] 72.5
```

```r
weighted.mean(x, w)
```

```
## [1] 72.5
```
